

Arhitectura calculatoarelor

Curs 10

10. Execuție speculativă și predicția ramificărilor

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

Obiective

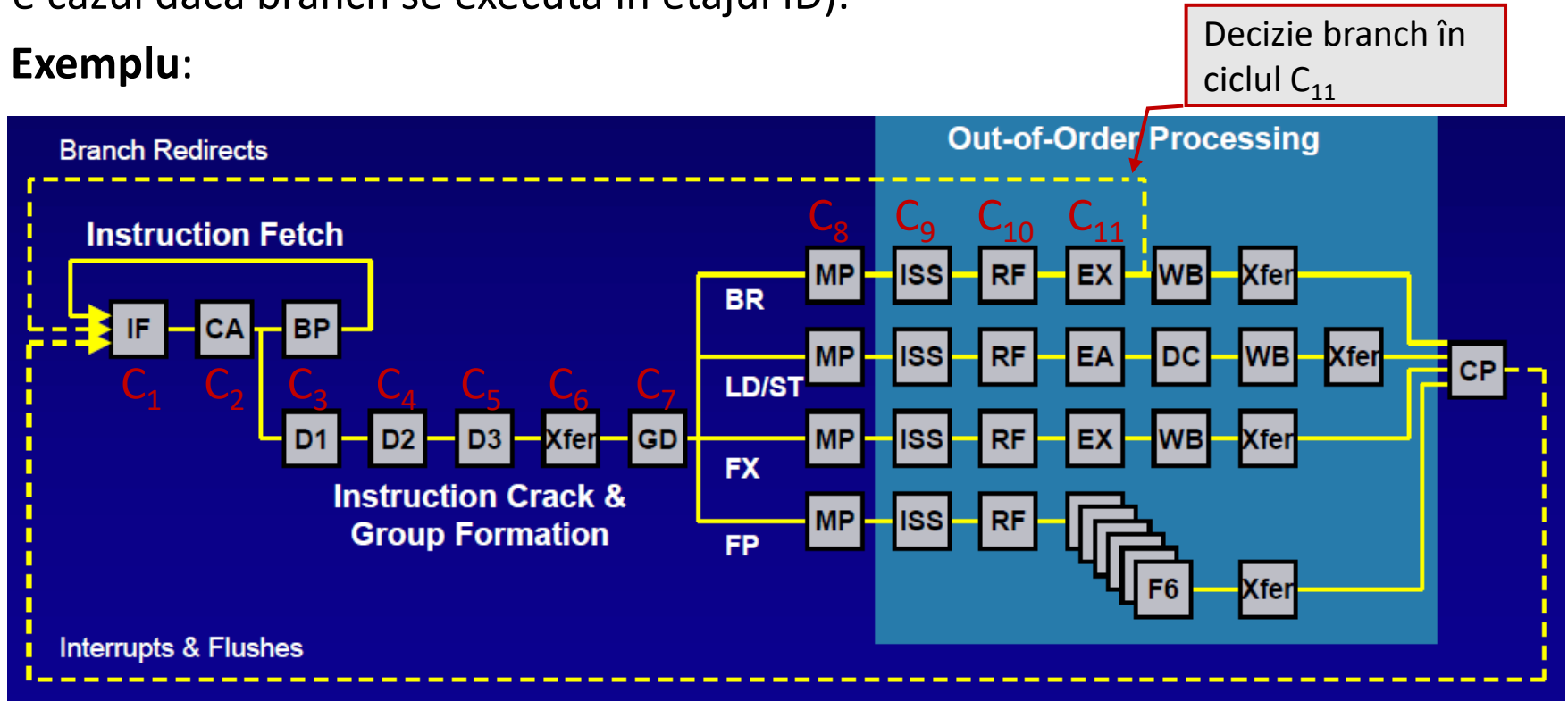
1. Execuția speculativă
2. Predicția salturilor
 - 2.1 Predicția statică
 - 2.2 Predicția dinamică
3. Tomasulo speculativ

10.1 Execuția speculativă

10.1.1 Motivație

- Dezavantaj Tomasulo: **Blocarea execuției instrucțiunilor** până la finalizarea branch-urilor anterioare, pentru a elimina hazardurile de control.
- Apar **întârzieri mari pentru arhitecturi în care branch-urile se finalizează târziu** (nu e cazul dacă branch se execută în etajul ID).

Exemplu:



IBM Power4 Pipeline: <http://ixbtlabs.com/articles/ibmpower4>

Obiectiv

- Lansarea în execuție, chiar dacă nu se știe decizia branch-urilor anterioare.

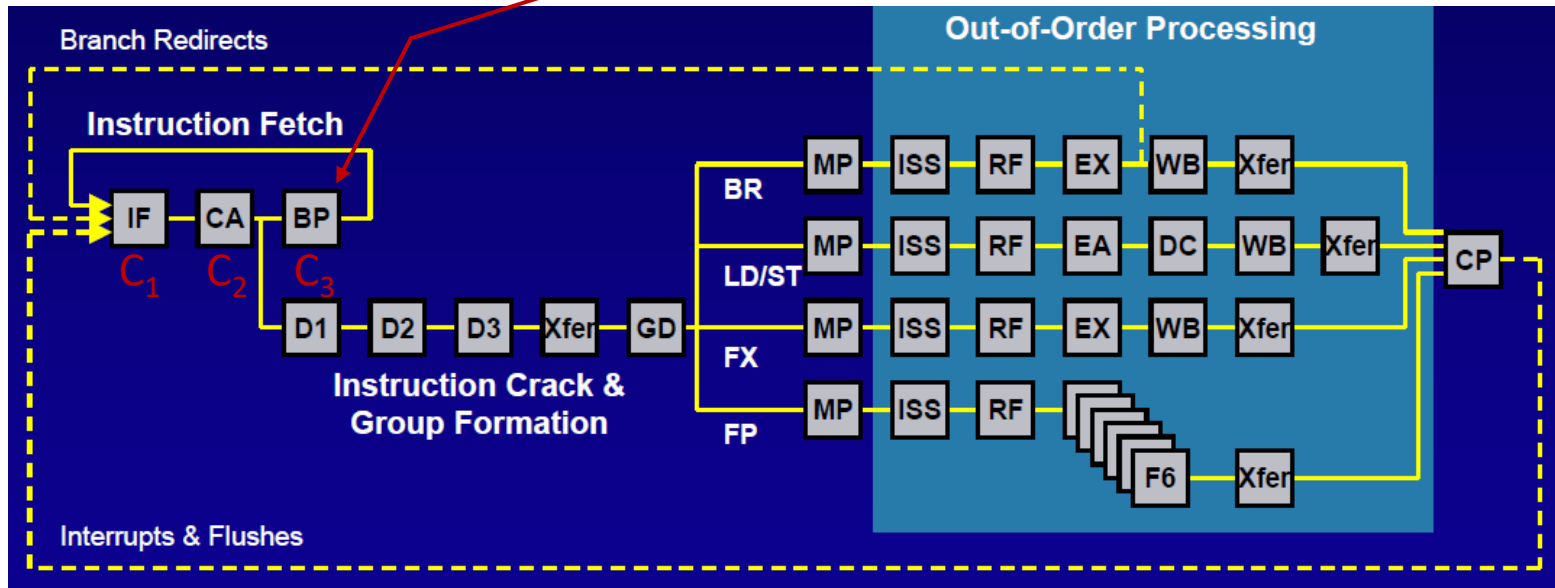
10.1 Execuția speculativă

10.1.2 Soluție bazată pe 3 elemente:

1. **Planificare dinamică** – instrucțiunile se execută odată ce au operanzi disponibili indiferent dacă sunt în așteptare instrucțiuni anterioare => execuție out-of-order, deoarece nu se mai finalizează în ordinea din program. (studiat la Tomasulo)
2. **Predicția dinamică a branch-urilor** – se presupune o anumită decizie de salt (taken/not taken) înainte să se știe decizia exactă. Se realizează în primele etaje => reduce din posibile penalități.

Branch Prediction - BP

Ex:



IBM Power4 Pipeline: <http://ixbtlabs.com/articles/ibmpower4>

3. **Speculație** – instrucțiunile ulterioare unui salt nu se blochează până la decizia acestuia, ci se execută în conformitate cu predicția => se iau măsuri de anulare a efectului lor în cazul în care, ulterior, decizia de salt diferă de predicție.

10.2 Predicția salturilor

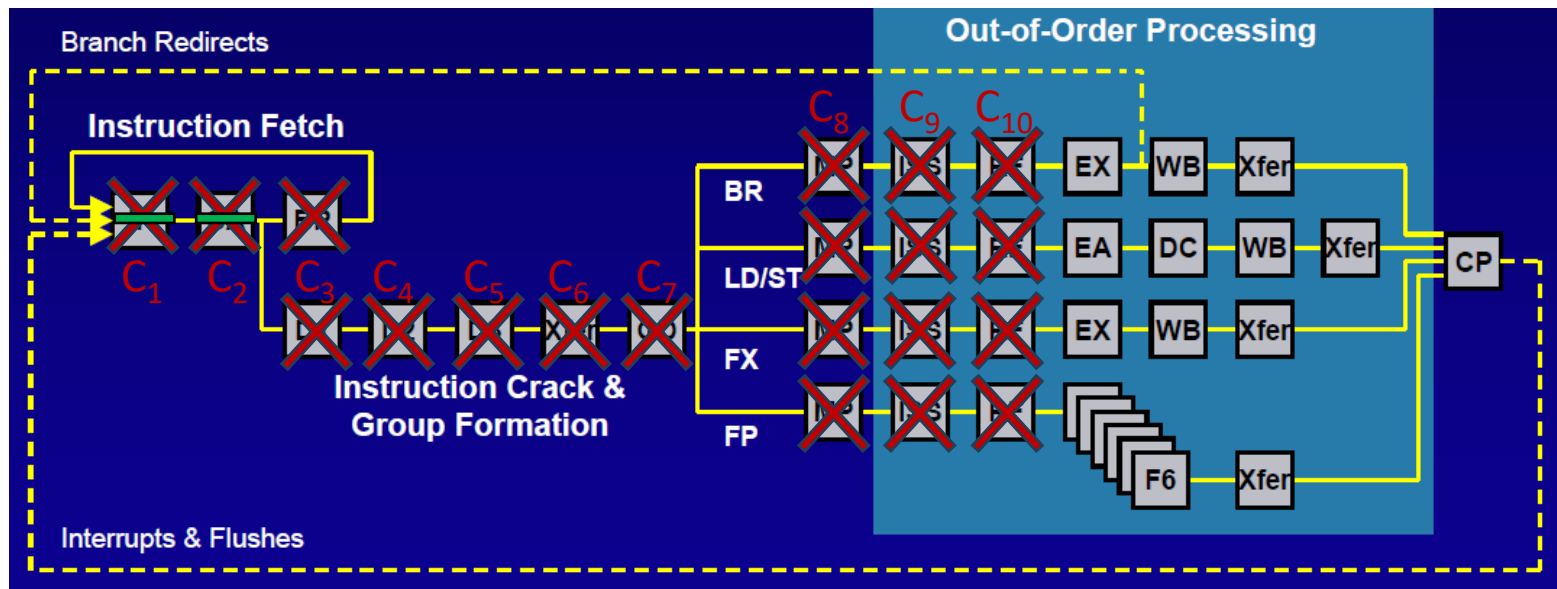
Pași:

1. Se prezice direcția taken/not taken (de regulă în etajul IF sau ID).
2. Se încarcă instrucțiuni în pipeline de la adresa corespunzătoare predicției.
3. După execuția branch (când se știe decizia reală):
 - Dacă predicția este corectă => nu apare penalitate;
 - Dacă predicția este incorectă => apare penalitate (**misprediction penalty**): se elimină din pipeline instrucțiunile încărcate greșit.

Notă: Un predictor este eficient dacă are rată mică de predicție greșită.

Ex: Posibile penalități de predicție BP la IBM Power4:

- Dacă **Branch Prediction = taken** => se elimină instrucțiunile din C_1 și C_2 ;
- Dacă **Branch Prediction $\neq$ decizia branch** => se elimină instrucțiunile din $C_1 - C_{10}$.



10.2.1 Predicția statică

- Varianta₁: Algoritmul **Always not taken**: nu se face salt (rată succes: 30–40%).
- Varianta₂: Algoritmul **Always taken**: întotdeauna se face salt (rată succes: 60–70%).
- Varianta₃: Realizată pe baza semnului câmpului offset al instrucțiunii branch (offset < 0 ⇔ salt înapoi – backward; offset > 0 ⇔ salt înainte – forward).
 - Ex: Algoritmul **Backward Taken Forward Not taken** (BTFN): dacă offset < 0 se face salt, altfel nu se face salt (rată succes: 65%).
- Varianta₄: **Realizată de compiler** pe baza analizei programului folosind biți dedicați (*hint bits*) din instrucțiune.
- Varianta₅: **Realizată de programator** prin macrouri dedicate (*pragmas*) în program.

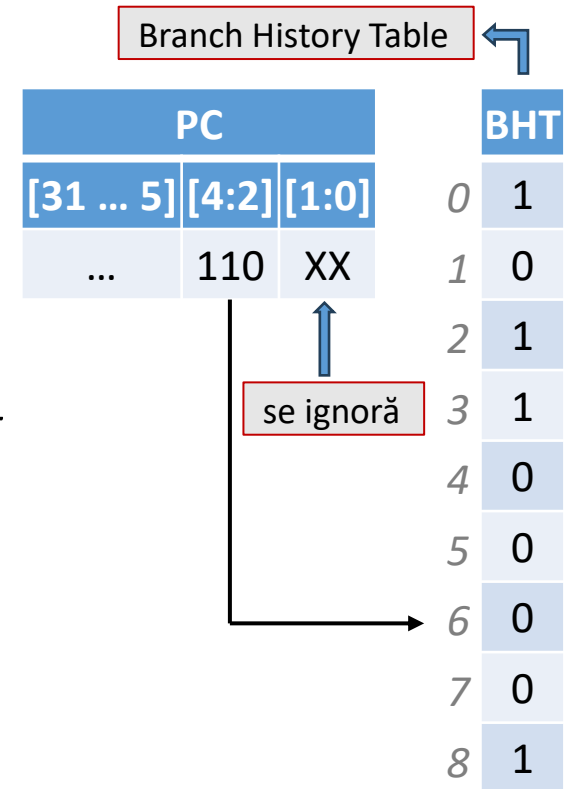
10.2.2 Predicția dinamică

- **Realizată cu unități hardware** dedicate care decid direcția (taken/not taken) în timpul execuției.
- **Strategie**: Direcția se stabilește pe baza deciziilor anterioare din timpul execuției: are la bază un **algorithm de învățare** care își modifică dinamic predicția în funcție de comportamentul în timp al fiecărei instrucțiuni de branch.

10.2.2 Predicția dinamică (cont)

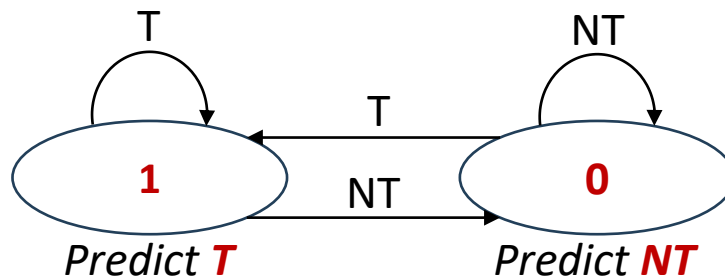
10.2.2.1 Predictorul cu 1 bit

- Cel mai simplu de implementat.
- Stochează un tabel (BHT) cu 1 bit de istorie pentru fiecare branch. Dimensiunea tabelului poate varia.
- Algoritm:
 - În faza de predicție: bit = 0 => fără salt (predict *Not Taken* – NT); bit = 1 => cu salt (predict *Taken* – T).
 - În faza de finalizare branch: se actualizează bitul corespunzător cu ultima decizie => predicția viitoare va fi în direcția ultimului branch încheiat.



Finite State Machine

Diagrama FSM de funcționare pentru 1 bit de predicție



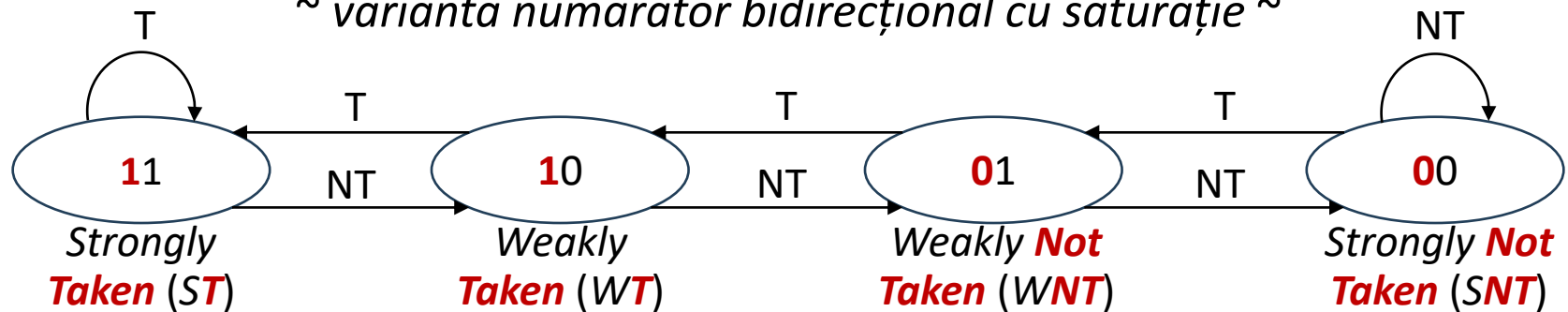
T = Taken
NT = Not Taken

Obs: Dacă mai multe instrucțiuni folosesc aceeași locație scade eficiența.

10.2.2 Predicția dinamică (cont)

10.2.2.2 Predictorul cu 2 biți

Diagrama FSM de funcționare pentru 2 biți de predicție
~ varianta numărător bidirecțional cu saturație ~



Definiție: Numărător cu saturație = dacă atinge valoarea maximă 11_2 nu se mai incrementează, iar dacă atinge valoarea minimă 00_2 nu se mai decrementează.

Ex: `sum=0; // suma elementelor unei matrice a[10][10]`

```
for (i=0; i<10; i++)
    for (j=0; j<10; j++)
        sum += a[i][j];
```



i	0						1					
j	0	1	2	...	9	10	0	1	2	...	9	10
BP _{for(j)}	WT	ST	ST	ST	ST	ST	WT	ST	ST	ST	ST	ST
Miss						X						X

Presupunem $BP_{for(j)} = WT$ inițial.

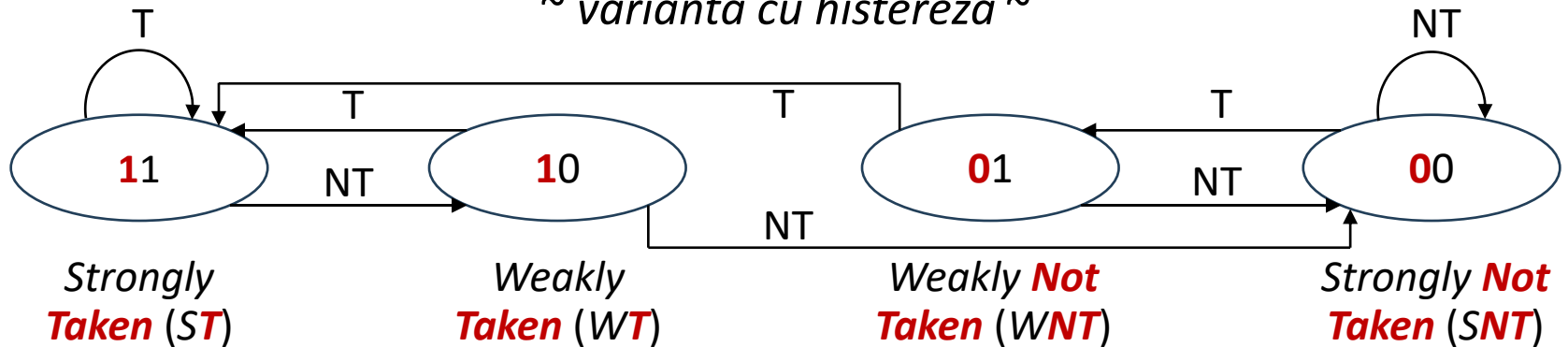
Rată succes $BP_{for(j)} : 9/10 \Rightarrow 90\%$.

- Obs:**
1. Poate tolera o predicție greșită fără să schimbe direcția.
 2. Două decizii în direcția opusă schimbă întotdeauna predicția.

10.2.2 Predicția dinamică (cont)

10.2.2.3 Predictorul cu 2 biți (cont)

Diagrama FSM de funcționare pentru 2 biți de predicție
~ varianta cu histereză ~



- Obs:**
1. Păstrează ideea de saturație.
 2. Două decizii în direcția opusă schimbă întotdeauna predicția și saturează invers.
 3. Păstrează rata de succes ca la numărătorul bidirecțional cu saturație.

10.2.2 Predicția dinamică (cont)

10.2.2.4 Concluzii

- Dacă predicția se face în ID și predicția = *Taken* atunci se anulează instrucțiunea din IF și se sare la adresa de branch.
- Efectele finalizării instrucțiunii de branch:
 1. Dacă predicția a fost eronată: **anulează instrucțiunile speculative** și reia lansarea instrucțiunilor de la adresa corectă => penalizare.
 2. **Actualizează predictorul** corespunzător din tabelul de predictor (Branch History Table – BHT) conform cu decizia curentă.

10.2.2.5 Posibilitatea mutării BP în etajul IF

Î: Unde este eficient să amplasăm predicția?

R: În primele etaje pentru a evita penalitatea la predicție taken?

Observații:

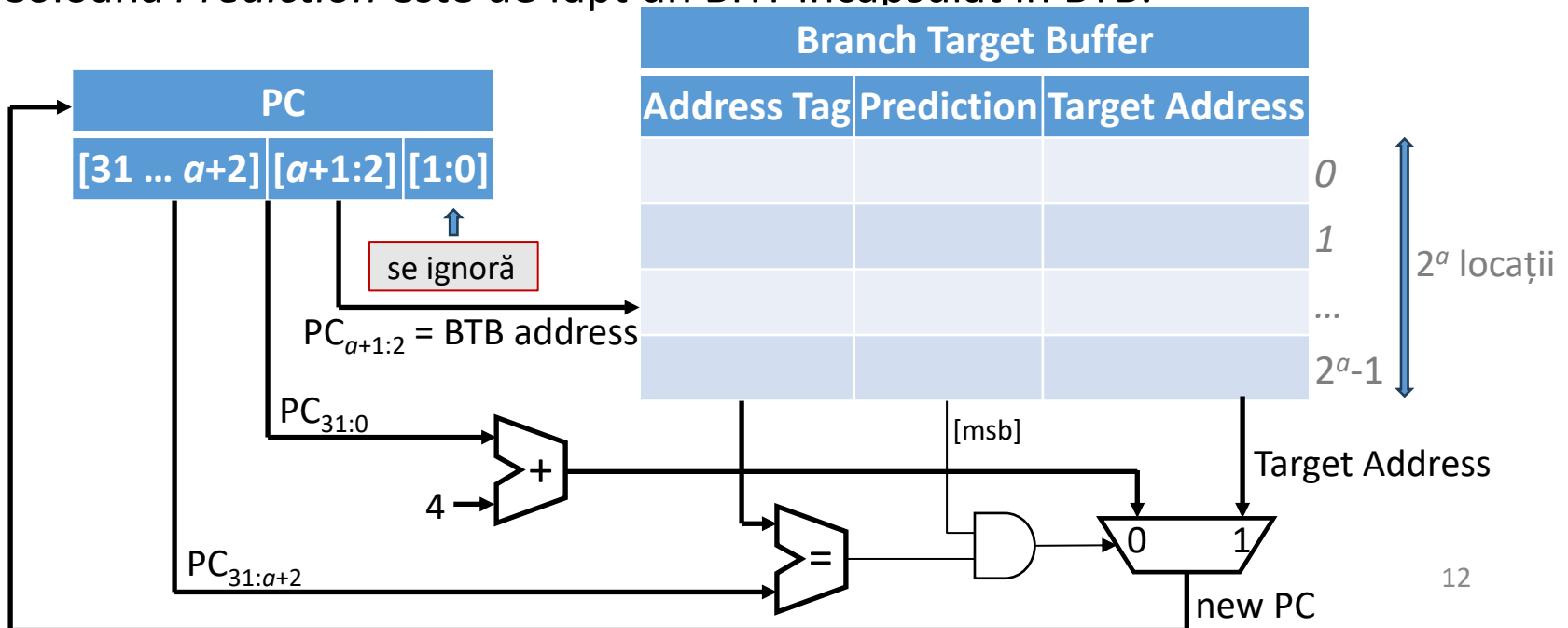
1. Predicția dinamică nu realizează testul, deci **nu necesită comparator**.
2. Predictorul folosește doar PC pentru a accesa **istoricul instrucțiunii**, deci se poate accesa în IF.
3. Predictorul trebuie să aibă efect numai la instrucțiuni de branch, deci este necesară o formă de **decodificare** a instrucțiunii.
4. Execuția speculativă are nevoie de **adresa de branch**.

Soluția: **Branch Target Buffer** (slide următor), care înlocuiește calculul adresei de branch, decodificarea instrucțiunii și oferă suport pentru predicție.

10.2.2 Predicția dinamică (cont)

10.2.2.6 Branch Target Buffer (BTB)

- Este o memorie cache care stochează adresa de salt (*Target Address*) și predicția (*Prediction*) pentru instrucțiuni de salt. Se amplasează, de regulă, în etajul IF.
- Inițial, BTB nu are date, dar pe parcurs **se completează doar cu instrucțiuni de salt**.
- Instrucțiunile sunt indexate de biții $PC_{a+1:2}$ și identificate pe baza biților mai semnificativi $PC_{31:a+2}$ care sunt salvați în *Address Tag*. $a = \log_2(\text{nr. rânduri în BTB})$. Dacă la instrucțiunea curentă $PC_{31:a+2} = \text{Address Tag}$ din BTB $\Rightarrow$ instrucțiunea este de salt (**înlocuiește decodificarea**).
- BTB **se actualizează când se finalizează o instrucțiune de salt**, cu datele acesteia.
- **Obs:** Coloana *Prediction* este de fapt un BHT încapsulat în BTB.



10.3 Tomasulo speculativ

10.3.1 Speculația

- Se realizează **în tandem cu predicția salturilor**: se execută instrucțiunile pe baza predicției, fără să se știe dacă este corectă.
- **Obs**: Instrucțiunile ajung în *Instruction Queue* **în ordinea de execuție speculativă**, conform cu Branch Prediction (BP) => după un Jump sau un Branch cu BP=taken, apar instrucțiunile de la adresa de salt, altfel se succed în ordinea secvențială.

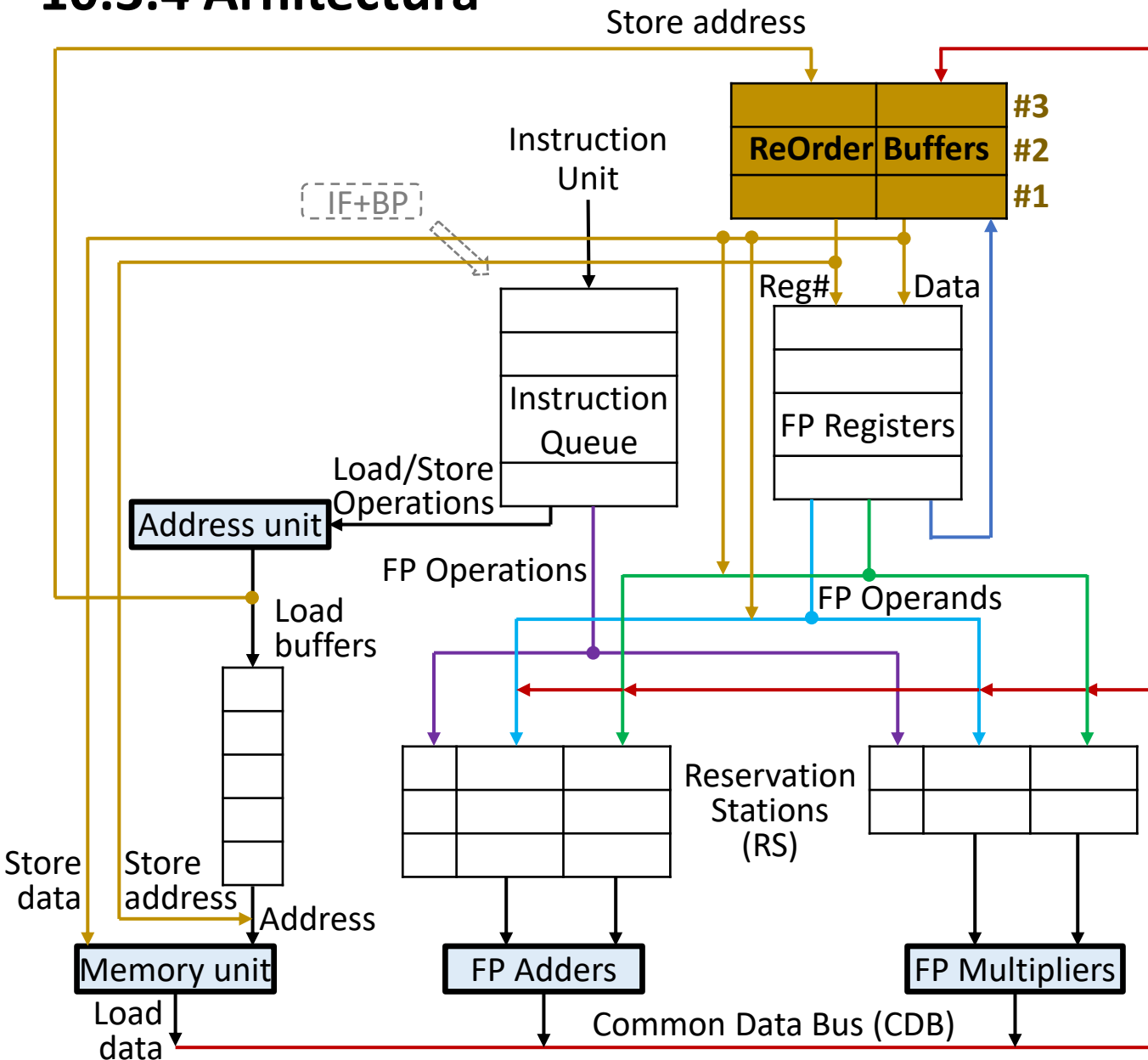
10.3.2 Mecanism

- În faza *Write-Result*, **rezultatul unei instrucțiuni** este stocat temporar într-un buffer adițional și **nu se scrie în registru/memorie până la validarea predicției** în baza căreia s-a executat.
- După validarea predicției (prin finalizarea instrucțiunii de branch) instrucțiunea nu mai este speculativă și va putea intra în faza de *Commit*, în care **se scrie rezultatul în registru/memorie, în ordinea din program** => chiar dacă fazele *Execute* și *Write-Result* se finalizează în altă ordine, instrucțiunile apar reordonate secvențial în buffer-ele adiționale – **ReOrder Buffers (ROB)**.
- Tomasulo speculativ = **in-order issue, out-of-order execute, in-order commit**.

10.3.3 Aplicabilitate

- Dezvoltat inițial pentru Power PC 603/604, MIPS R10000/R12000, Intel Pentium II/III/4, AMD K5/K6/Athlon, etc.
- Cunoaște diverse variante de implementare pe majoritatea procesoarelor actuale.

10.3.4 Arhitectura



Elemente specifice

- Setul **ROBs** este organizat ca o coadă (FIFO) la care instrucțiunile sunt "adăugate", chiar la lansare (*Issue*), în ordinea din *Instr.Queue*.
- **ROBs** preiau rolul buffer-elor Store => nu mai sunt necesare buffere Store.
- Fiecare **ROB** are ID care înlocuiește rolul ID-urilor stațiilor RS și a buffer-elor Load => pe CDB se folosesc numai etichete ale ROBs.
- Toate rezultatele sunt difuzate prin CDB către stațiile RS și setul ROBs.
- Scrierea în registre/memorie are loc din ROB în faza de *Commit*.

10.3 Tomasulo speculativ

10.3.5 Structura ROB

Un ROB conține mai multe câmpuri de informație:

- **Tipul instrucțiunii** – în funcție de care se interpretează celelalte câmpuri:
 - Branch – nu are rezultat și nici destinație;
 - Store – destinația este o adresă de memorie;
 - Operație cu registru (aritmetică-logică sau load) – destinația este un registru.
- **Destination** – poate conține:
 - Index de registru – la operații aritmetice-logice sau load;
 - Adresă de memorie – la store.
- **Value** – conține valoarea rezultatului de scris în registru/memorie la *Commit*.
- **Ready** – specifică dacă s-a încheiat execuția și se cunoaște rezultatul.
- **Speculative** – indică starea speculativă a instrucțiunii:
 - Speculative – instrucțiunea este speculativă, branch-ul nu s-a încheiat;
 - Confirmed – branch-ul s-a încheiat, predicția a fost corectă;
 - Not confirmed – branch-ul s-a încheiat, predicția a fost incorectă.

10.3 Tomasulo speculativ

10.3.6 Pași

1. **Issue** – Lansarea instrucțiunii din vârful cozii de instrucțiuni (*Instruction Queue*).
 - Instrucțiunea se lansează dacă există stație RS sau buffer Load liber, și ROB liber, altfel se face **Stall** (**hazard structural**).
 - Se alocă stație RS sau buffer Load și un ROB. La instrucțiune Store se alocă doar ROB.
 - Operanzii disponibili sunt citați din blocul de registre(primar) sau din ROB(secundar).
 - Operanzii indisponibili sunt înlocuiți cu ID-ul ROB-ului care îi prelucrează.
 - Redenumirea registrului (*register rename*): dacă instrucțiunea scrie un registru, acesta se redenumeste cu ID-ul ROB, până la finalizare (**rezolvă hazardurile WAW și WAR**). **Notă:** Stația RS/buffer-ul Load va folosi ID-ul ROB pentru difuzarea rezultatului pe CDB.
2. **Execute** – Execuția unei instrucțiuni după lansare.
 - Operanzii indisponibili sunt așteptați pe CDB pe baza ID-ului de broadcast.
 - Când toți operanzii sunt disponibili (**rezolvă hazardurile RAW**) se execută operația.
 - Un load are 2 pași de execuție: calcul adresă + citirea din memorie.
 - Un store realizează doar calculul adresei (scrierea în memorie are loc în *commit*).
3. **Write result** – Difuzarea rezultatului pe CDB după execuție.
 - Rezultatul și ID-ul se difuzează pe CDB către stațiile și ROB-ul care îl așteaptă. La nivelul ROB rezultatul se stochează în *Value*. Se poate difuza un rezultat pe ciclu.
 - Un store nu difuzează, doar așteaptă valoarea și apoi o stochează în câmpul *Value*.
 - Stația sau buffer-ul Load se eliberează.

10.3 Tomasulo speculativ

10.3.6 Pași (cont)

4. **Commit** – Finalizarea instrucțiunii (înlocuiește *Write-Back*).

- Are loc când instrucțiunea este în capul ROB și cunoaște rezultatul. Există 3 cazuri în funcție de tipul instrucțiunii:
 - Instrucțiune de branch – dacă predicția a fost incorectă se eliberează pozițiile următoare din ROB (**Tail** ← **Head**), se renunță la redenumirile corespunzătoare și se reia execuția cu adresa corectă; dacă predicția a fost corectă execuția continuă normal;
 - Instrucțiune de store – se scrie valoarea din câmpul *Value* în memorie;
 - Operație cu registru – se scrie valoarea din câmpul *Value* în registru.
- După terminarea unei instrucțiuni poziția din ROB se eliberează. Se renunță la redenumirea registrelor asociate cu poziția ROB eliberată.

10.3.7 Rezolvarea hazardului la memorie – **Dynamic memory disambiguation**

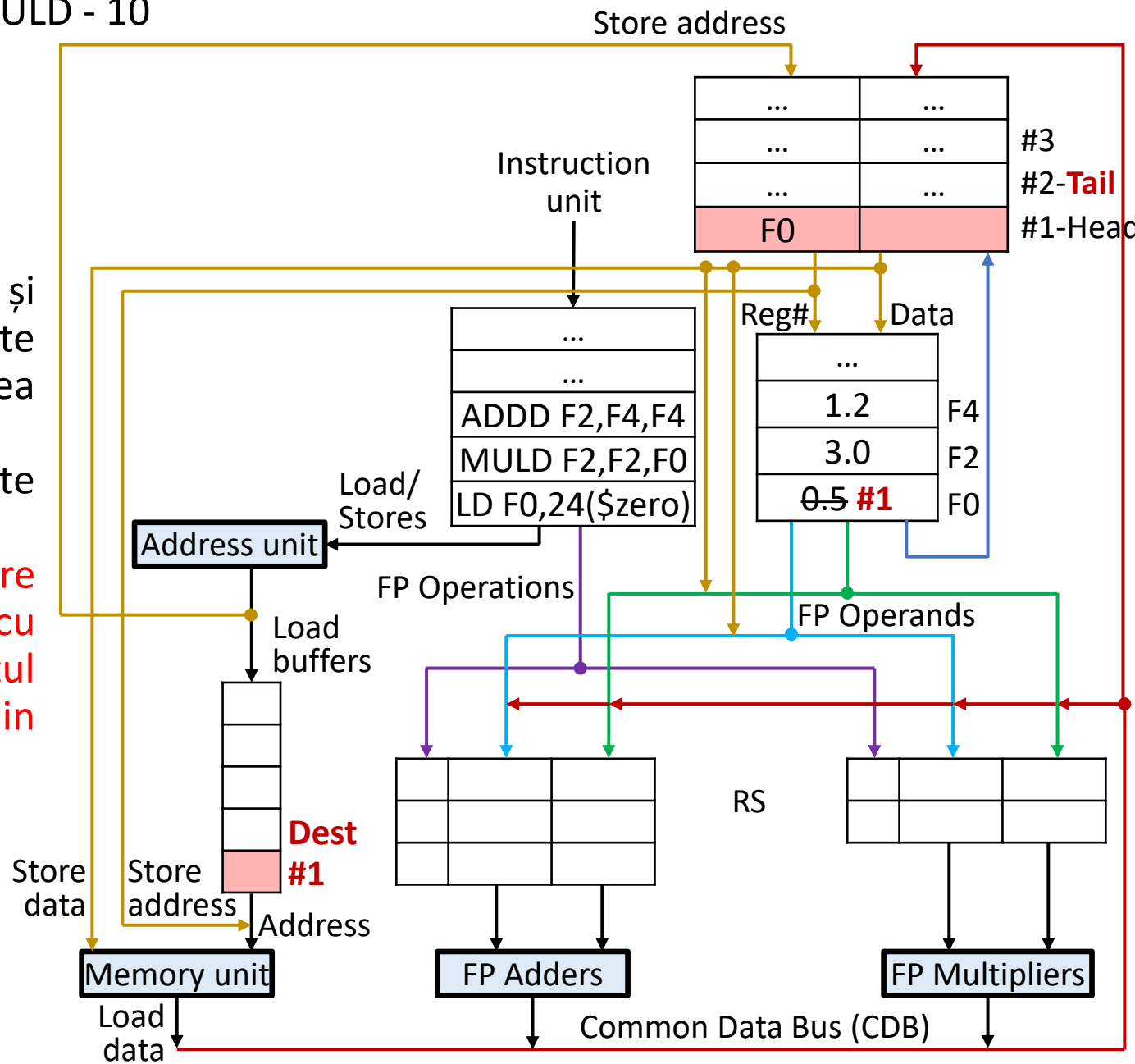
- **Obs:** Nu apare hazard între load-uri succesive => se pot executa în orice ordine din buffer-ele Load.
- Hazardurile WAW și WAR la memorie nu apar fiindcă scrierile în memorie se fac în ordine de către ROBs, în consecință, toate citirile și scrierile anterioare sunt finalizate.
- Hazardurile RAW la memorie se rezolvă prin 2 restricții:
 1. O adresă de load se calculează în *Address Unit*, numai după ce s-au calculat adresele de store anterioare, în ordinea din program;
 2. Nu se permite pasul 2 al execuției unui load (citirea din memorie), dacă în ROBs există o instrucțiune store anterioară (către **Head**), cu adresă identică.

Exemplu₁ Timpi execuție [cicluri clk]: LD - 8 (1 calcul adresă + 7 citire memorie, (cont) ADDD - 4, MULD - 10

- Ciclu 1:
- Issue LD

- Evenimente:
- Se alocă ROB #1 și buffer Load (marcate cu roșu) pentru citirea F0 din memorie.
 - F0 se redenumțește temporar la #1.

Obs: Buffer-ul Load are Dest=#1, adică ID-ul cu care va difuza rezultatul pe CDB după citirea din memorie.

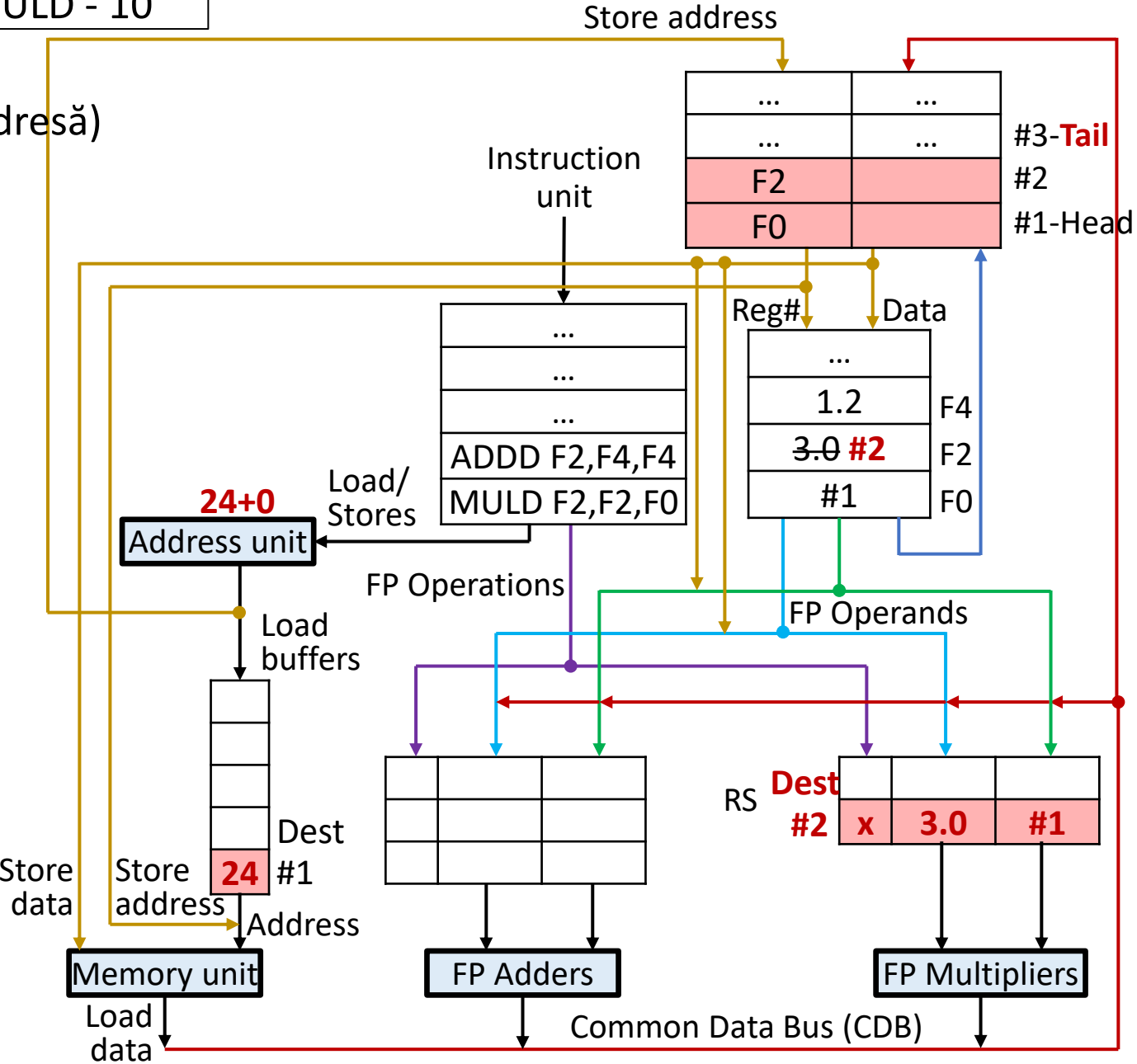


Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

- Ciclu 2:
- **Ex-pas₁** LD (calcul adresă)
 - **Issue** MULD

- Evenimente:
- Se alocă ROB #2 și stație RS (marcate cu roșu) pentru MULD.
 - F2 se redenumeste temporar la #2.

Obs: Stația pt. MULD primește valoarea 3.0 de la reg. F2 (apoi F2 este redenumit), iar de la F0 primește #1, deci verifică dacă ROB #1 are valoare. În lipsa ei, folosește ID-ul #1 și va aștepta valoarea pe CDB.

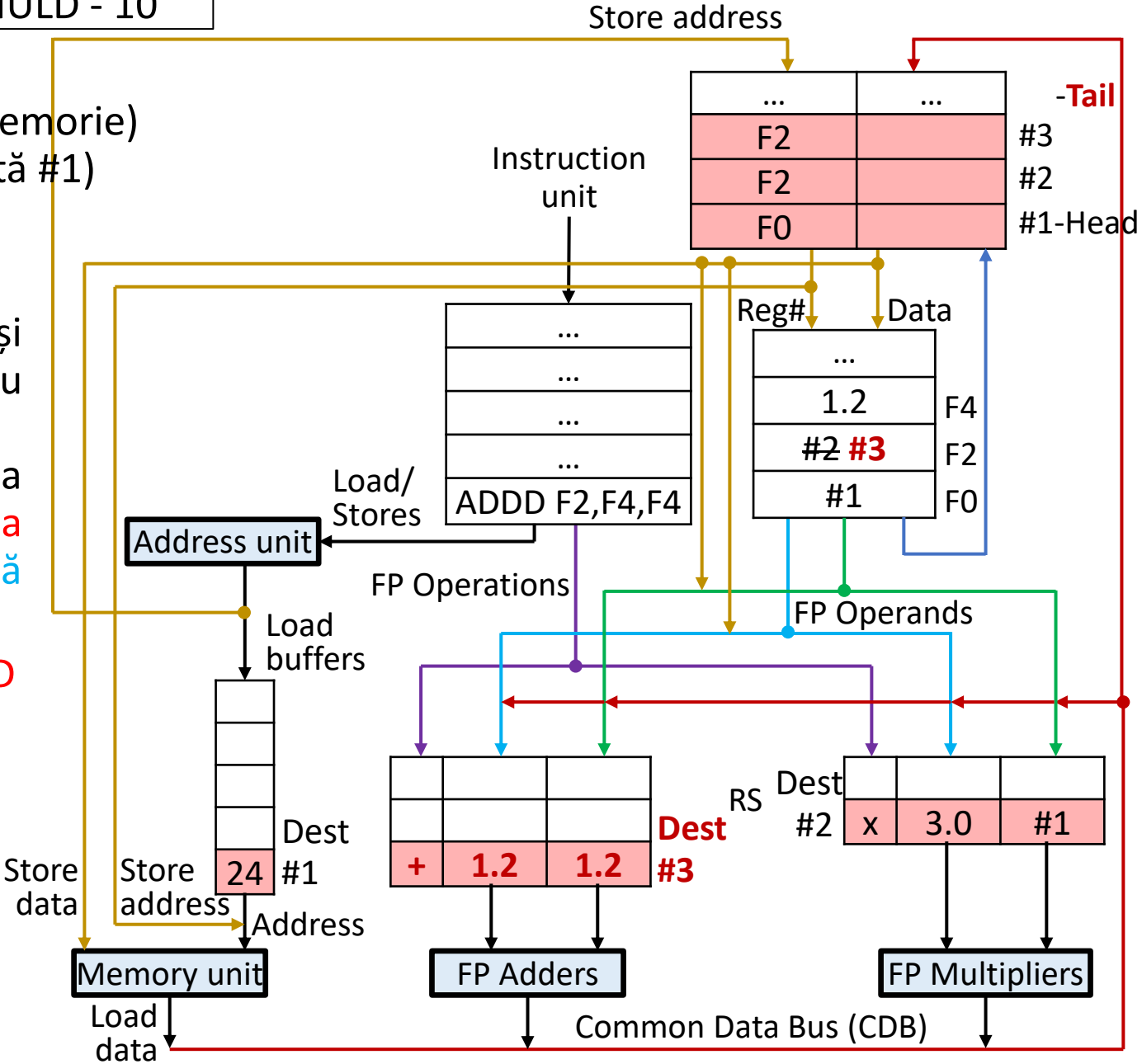


Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

- Ciclu 3:
- **Ex-pas₂** LD (citire memorie)
 - Stall MULD (așteaptă #1)
 - **Issue** ADDD

- Evenimente:
- Se alocă ROB #3 și stație RS (marcate cu roșu) pentru ADDD.
 - F2 se redenumesc la #3 => **va primi ultima scriere în el (elimină hazardul WAW)**.

Obs: Stația pt. ADDD primește 1.2 de la F4.

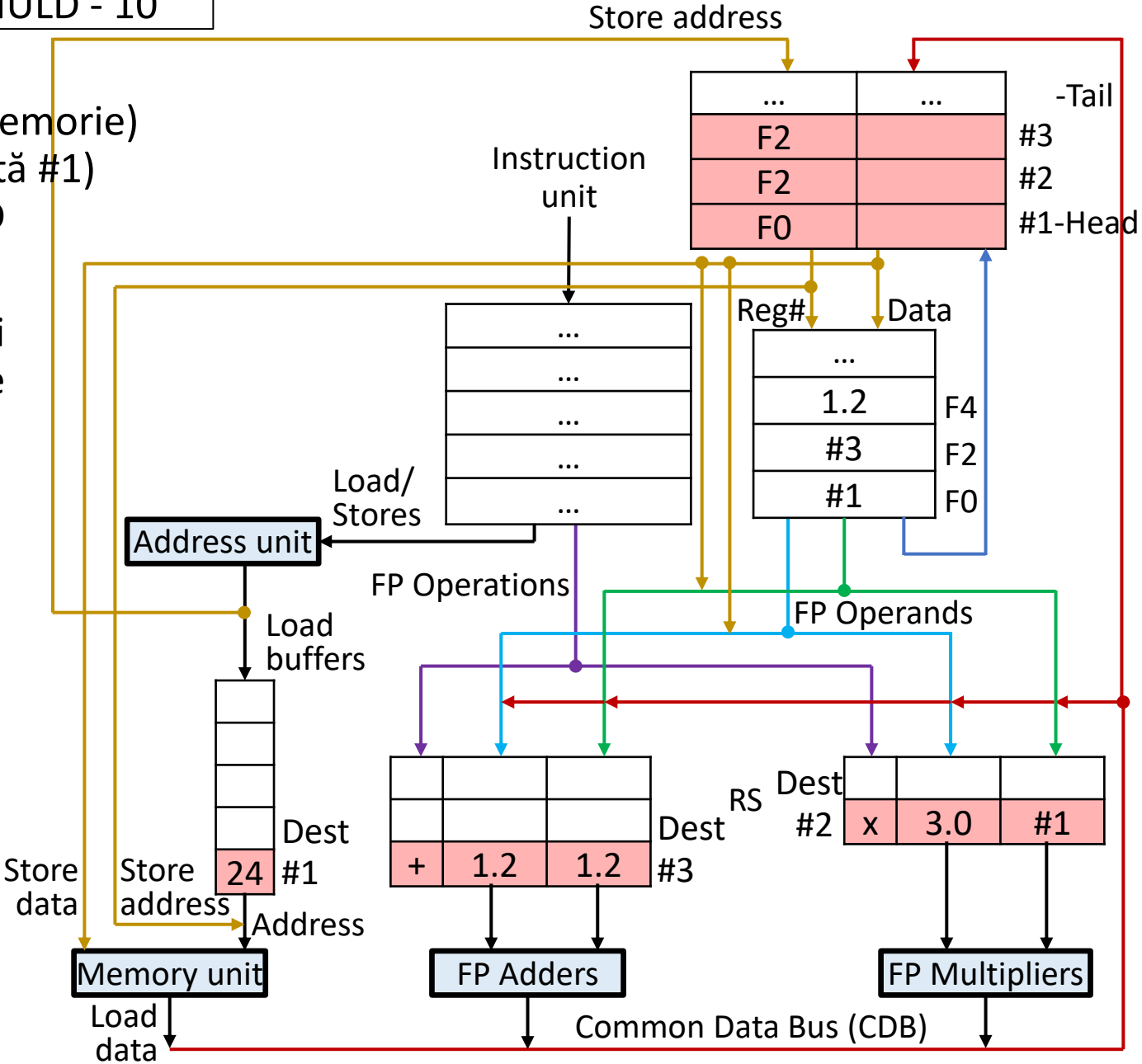


Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

- Ciclu 4:
- **Ex-pas₂** LD (citire memorie)
 - Stall MULD (așteaptă #1)
 - **Start execute** ADDD

Evenimente:

- Adunarea are toți operandii => începe execuția ADDD.



Exemplu₁ (cont)

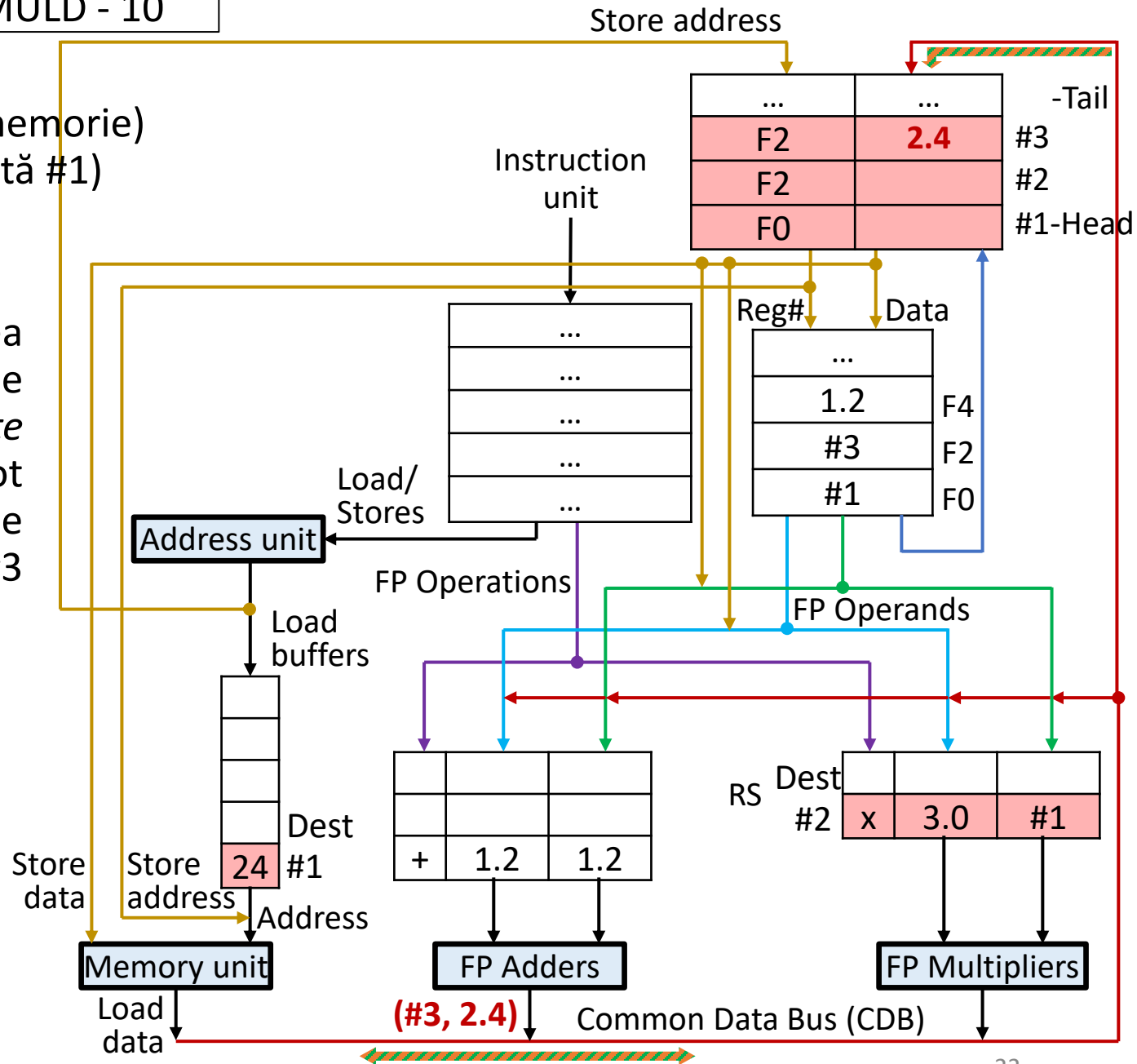
Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 8:

- **Ex-pas₂** LD (citire memorie)
- Stall MULD (aşteaptă #1)
- **Write result** ADDD

Evenimente:

- S-a finalizat adunarea și se difuzează **2.4** pe CDB cu ID-ul **#3** (*write result*) => peste tot (exceptând registrele FP) se înlocuiește #3 cu valoarea 2.4.
- Se eliberează stația.



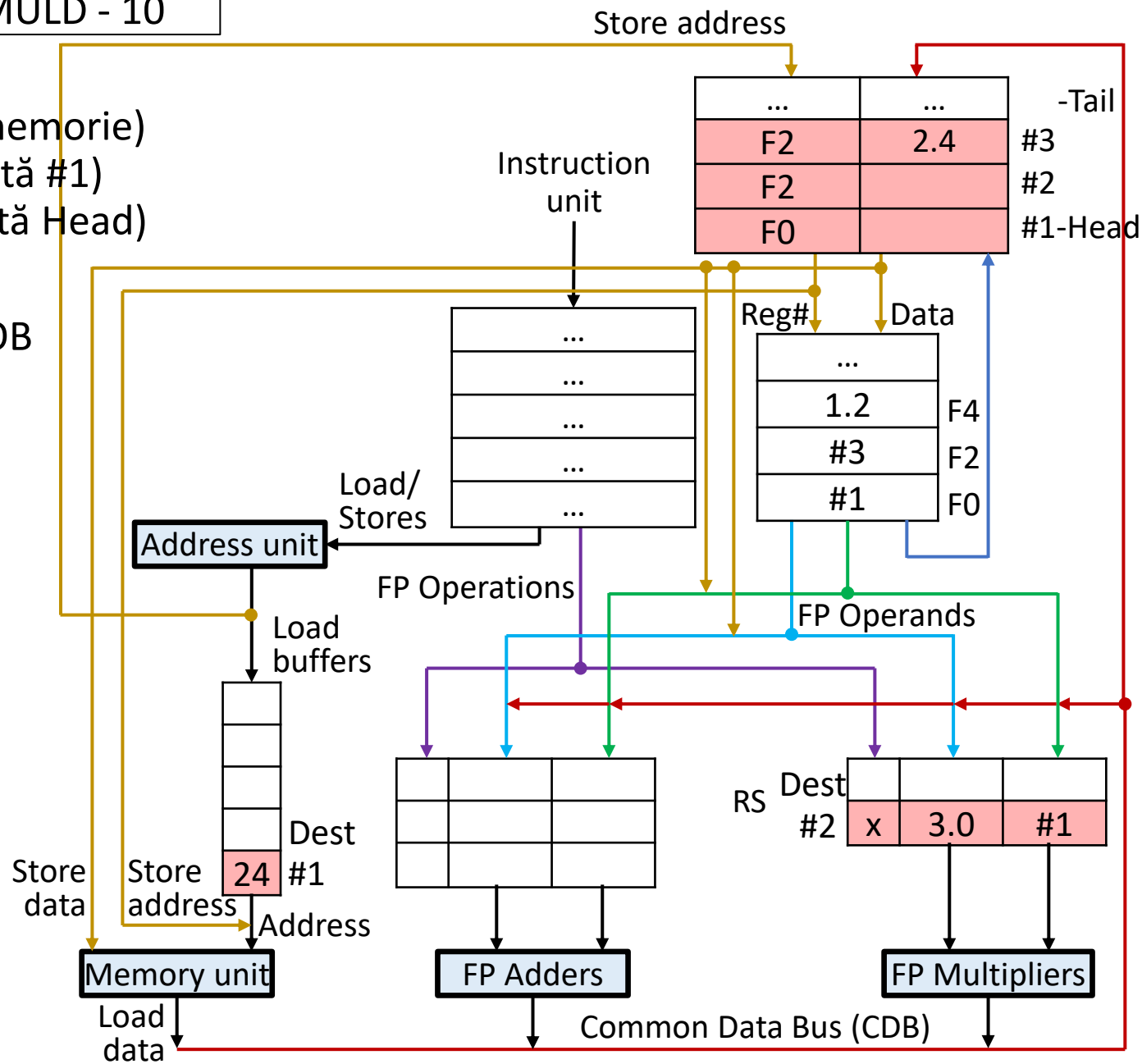
Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

Ciclu 9:

- **Ex-pas₂** LD (citire memorie)
- Stall MULD (așteaptă #1)
- Stall ADDD (așteaptă Head)

Evenimente:

- ADDD așteaptă în ROB finalizarea (*commit*) instrucțiunilor anterioare.



Exemplu₁ (cont)

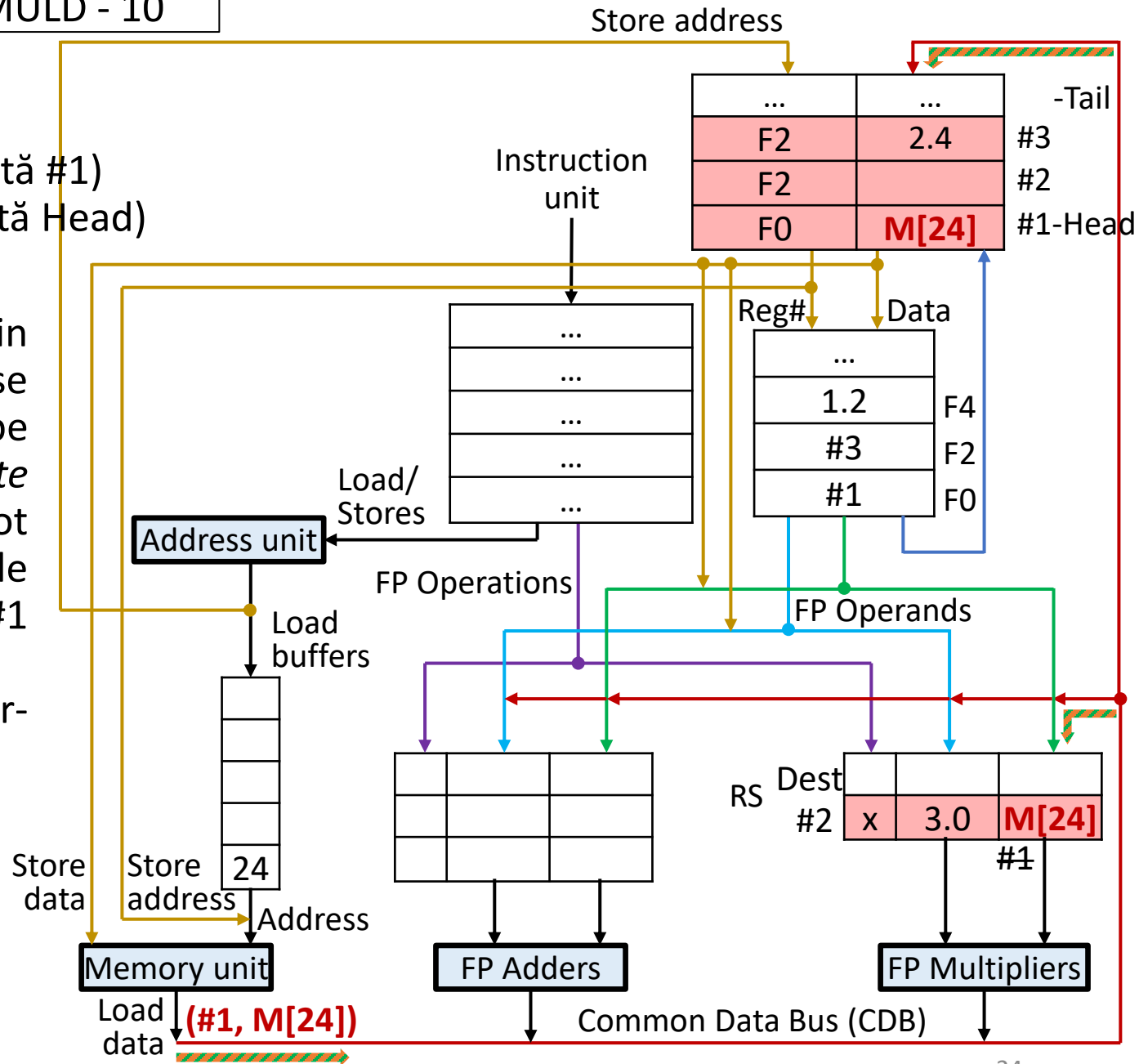
Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 10:

- **Write result** LD
- Stall MULDD (așteaptă #1)
- Stall ADDDD (așteaptă Head)

Evenimente:

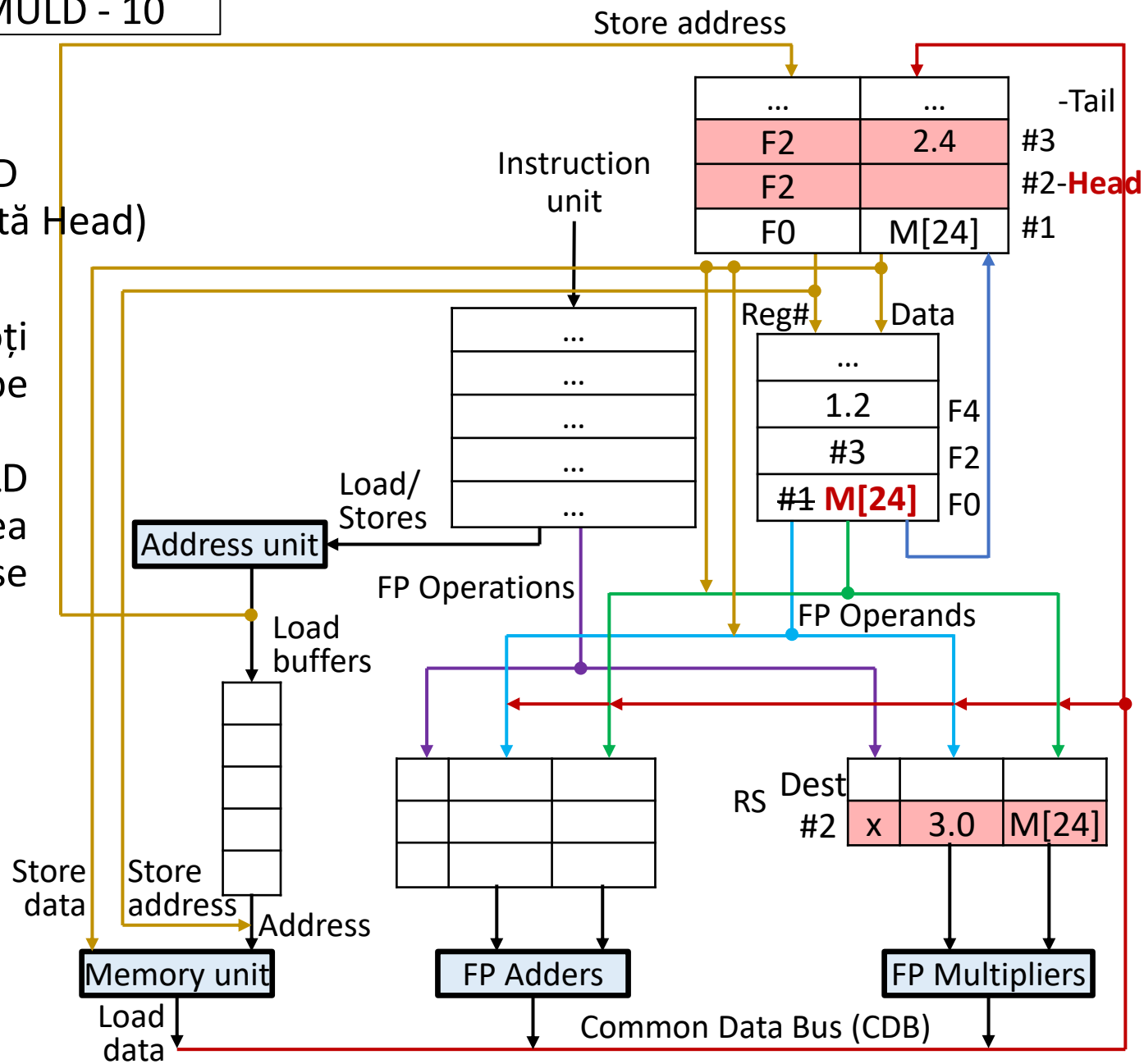
- S-a finalizat citirea din memorie și se difuzează **M[24]** pe CDB cu ID-ul **#1** (*write result*) => peste tot (exceptând registrele FP) se înlocuiește #1 cu valoarea M[24].
- Se eliberează buffer-ul.



Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

- Ciclu 11:
- **Commit** LD
 - **Start execute** MULD
 - Stall ADDD (așteaptă Head)

- Evenimente:
- Înmulțirea are toți operandii => începe execuția MULD.
 - Se finalizează LD (*commit*) cu scrierea rezultatului în F0 și se eliberează ROB #1.



Exemplu₁ (cont)

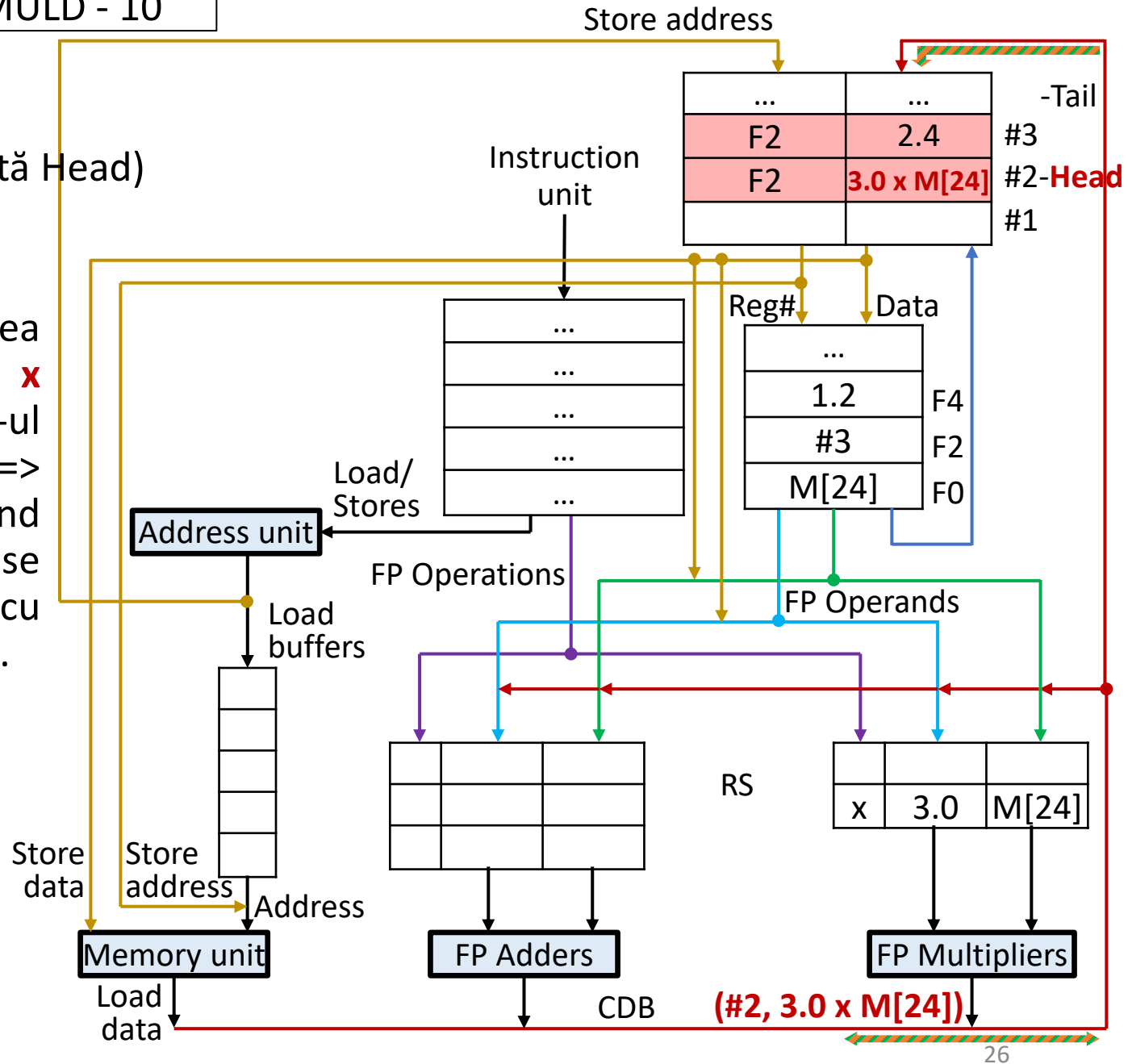
Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 21:

- Write result MULDD
- Stall ADDDD (așteaptă Head)

Evenimente:

- S-a finalizat înmulțirea și se difuzează **3.0 x M[24]** pe CDB cu ID-ul **#2** (*write result*) => peste tot (exceptând registrele FP) se înlocuiește #2 cu valoarea 3.0 x M[24].
- Se eliberează stația.



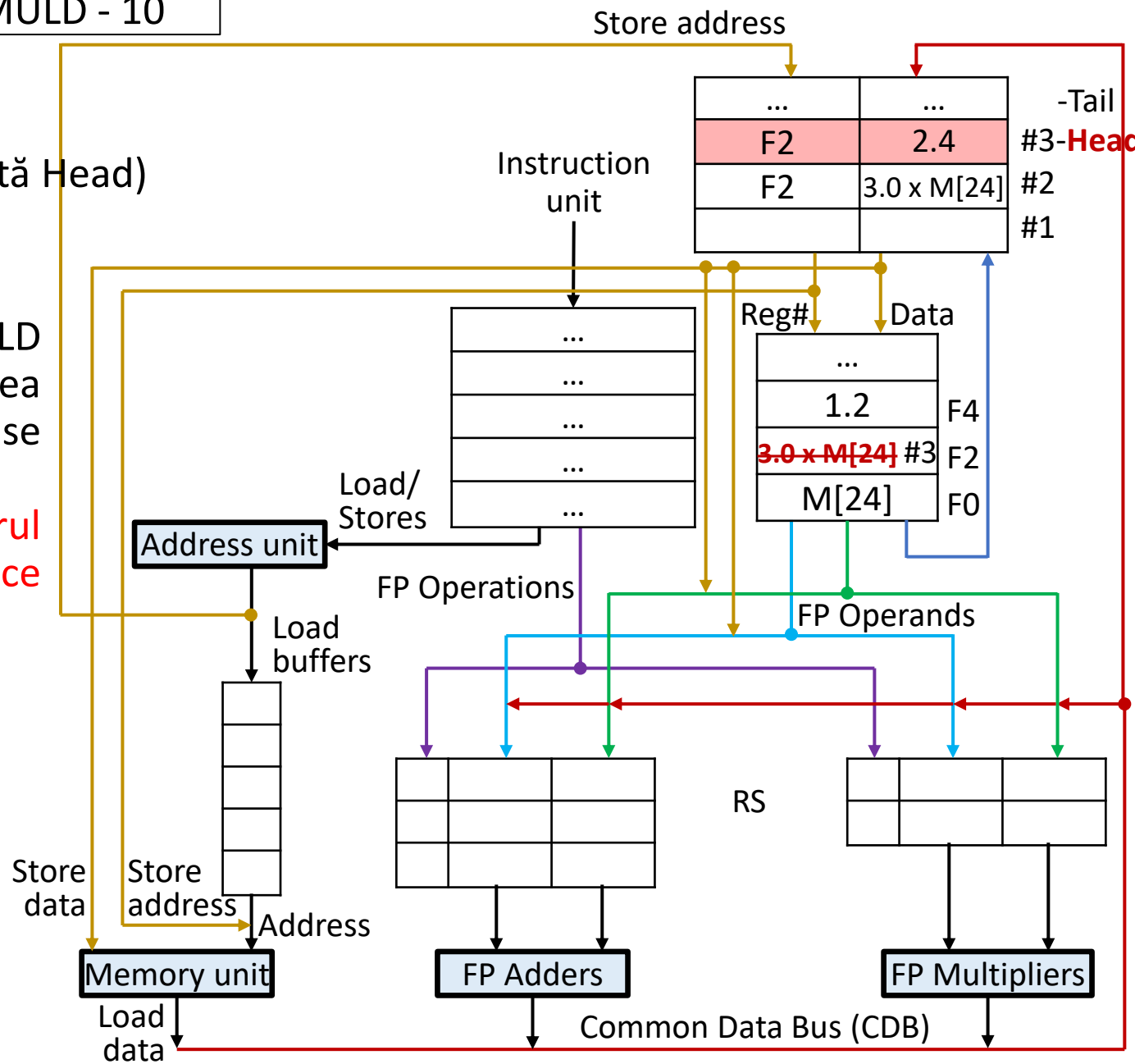
Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

- Ciclu 22:
- Commit MULD
 - Stall ADDD (așteaptă Head)

Evenimente:

- Se finalizează MULD (*commit*) cu scrierea rezultatului în F2 și se eliberează ROB #2.

Obs: Scrierea în registrul F2, eșuează deoarece este redenumit la #3.

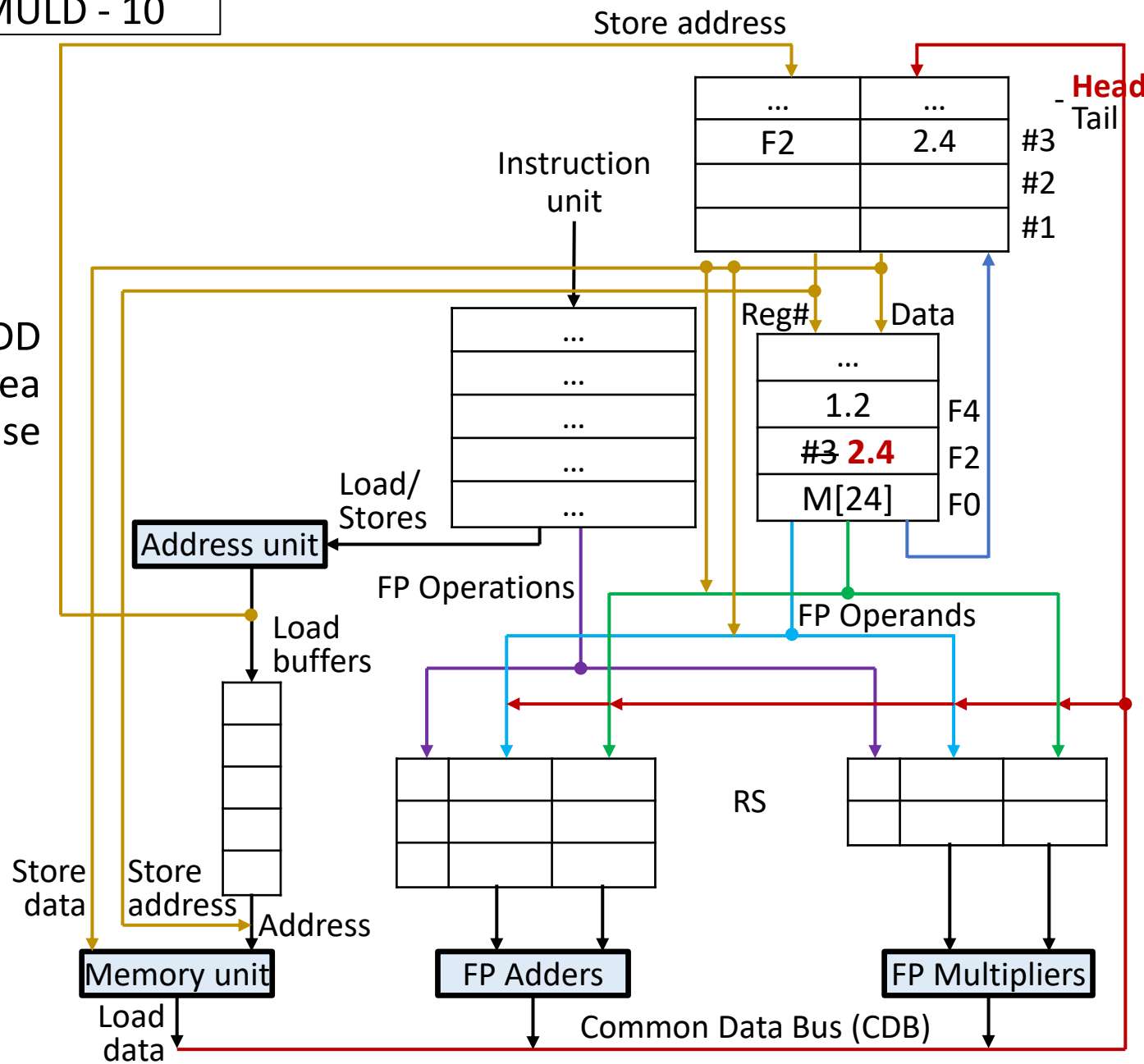


Exemplu₁ Timpi execuție: LD - 8,
(cont) ADDD - 4, MULD - 10

- Ciclu 23:
- **Commit** ADDD

Evenimente:

- Se finalizează ADDD (*commit*) cu scrierea rezultatului în F2 și se eliberează ROB #3.



10.3 Tomasulo speculativ

10.3.8 Planificarea execuției

Exemplu₂

Timpi de execuție:
LD - 2, ADDD/SUBD - 2,
MULD - 10, DIVD - 40

Specificații:

1. Presupunem că sunt un număr nelimitat de stații RS, buffere Load și ROBs => nu va fi hazard structural în prima coloană și la fiecare ciclu se face **Issue** la câte o nouă instrucțiune.
2. Toate coloanele (exceptie **Com**) au aceeași semnificație ca la Tomasulo (vezi exemplul din cursul anterior).

#	Instrucțiune	Tomasulo								Tomasulo speculativ							
		Str	Iss	RAW	sE	eE	Cdb	Wr		Str	Iss	RAW	sE	eE	Cdb	Wr	Com
1	LD F6 34(\$t2)		1		2	3		4			1		2	3		4	5
2	LD F2 45(\$t3)		2		3	4		5			2		3	4		5	6
3	MULD F0 F2 F4		3	2	6	15		16			3	2	6	15		16	17
4	SUBD F8 F6 F2		4	2	6	7		8			4	2	6	7		8	18
5	DIVD F10 F0 F6		5	3	17	56		57			5	3	17	56		57	58
6	ADDD F6 F8 F2		6	4	9	10		11			6	4	9	10		11	59

Obs:

1. Dacă nu apar branch-uri sau hazarduri la memorie, planificarea cu Tomasulo speculativ seamănă cu Tomasulo, exceptând coloana **Com**.
2. La o secvență fără branch-uri și hazard la memorie, Tomasulo speculativ poate înregistra câteva cicluri de ceas în plus.

↑
Ciclul
pentru
Commit

10.3 Tomasulo speculativ

10.3.9 Limite impuse speculației

- **Obs:** La un moment dat putem avea în ROB-uri mai multe branch-uri consecutive executate speculativ, ceea ce reprezintă un **avantaj în următoarele situații**, în care se evită întârzierile:
 1. Frecvență ridicată a instrucțiunilor de branch;
 2. Aglomerare de branch-uri în zone restrânse;
 3. Întârzieri mari ale unităților funcționale.
- Speculația poate reprezenta un **dezavantaj când apar excepții costisitoare** (ex. accesul eșuat la cache de nivel 2 poate dura mii de cicluri de ceas) și predicția nu este confirmată, în cele din urmă.

Soluția: Se impun **limite ale speculației atunci când apar excepții de cost ridicat** => execuția se oprește și se așteaptă confirmarea predicției înainte de tratarea excepției. Excepțiile cu cost redus (ex. accesul eșuat la cache de nivel 1) se execută speculativ.

Referințe

- [1] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 4.8, Ed. Morgan–Kaufmann, 2013.
- [2] J. L. Hennessy, D. A. Patterson, “Computer Architecture: A Quantitative Approach”, 5th edition, Cap. 3.6, 3.9, Anexa C2, Ed. Morgan–Kaufmann, 2011.
- [3] B. H. Juurlink, “Course 2 – ILP, DLP, TLP: Module 2.1 – Dynamic Exploitation of ILP: 2.1.10 – Hardware Based Speculation”, High-Performance and Embedded Computer Architecture. Online:
<https://youtu.be/mFtX28lH4O8?list=PLeWkeA7esB-PcOTrTCvAsaCArnCMQkcNv>